AUDIT TECHNIQUE DU FIRMWARE

Revue complète du projet FirmwareGPS

Analyse du code, de l’architecture, de la sécurité, des tests et de l’aptitude à une mise en service.

Projet : Fortico FirmwareGPS — ESP32 DOIT DevKit V1 / SIM7600

Dépôt : Fortuno-tech/FirmwareGPS

Date de revue : 6 novembre 2026

Périmètre observé : code source, configuration PlatformIO, tests présents et documentation technique disponible dans le dépôt.

Statut global : compilation réussie ; plusieurs bloqueurs de sûreté/sécurité et de validation avant déploiement.

1. Synthèse exécutive

> **Conclusion :** le dépôt constitue une base firmware structurée et compile pour la cible ESP32, mais les éléments examinés ne justifient pas une mise en flotte ou une validation de sécurité. Les commandes distantes de relais ne sont pas protégées contre le rejeu au niveau applicatif, un badge RFID partagé est ajouté automatiquement à chaque boîtier provisionné, et les tentatives réseau bloquantes s'exécutent dans la tâche qui porte aussi le cycle de contrôle. Les tests existants ne se construisent pas avec la configuration actuelle.

La revue identifie quatre sujets à traiter avant tout pilote opérationnel : (1) authentification, autorisation et anti-rejeu des commandes MQTT ; (2) suppression du badge RFID codé en dur ; (3) séparation stricte entre contrôle du relais et E/S réseau bloquantes ; (4) configuration et vérification effectives du Secure Boot et du chiffrement flash. Il faut également rétablir les tests, fermer le chemin de perte de télémétrie hors ligne et terminer ou désactiver explicitement la fonctionnalité OTA non intégrée.

Les résultats sont fondés sur le dépôt et une compilation locale. Aucun matériel, broker, certificat de production, configuration de programmation usine ou scénario HIL n'a été disponible ; l'état réel des périphériques et les ACL du broker ne sont donc pas certifiés.

2. Périmètre et méthode

- Lecture de la configuration PlatformIO, du point d'entrée, de l'orchestration système, des chemins MQTT, RFID, OTA, sécurité, stockage et ordonnanceur.
- Confrontation aux spécifications d'architecture (Document 02), MQTT (Document 08) et validation (Document 11) incluses dans `docs\`.
- Recherche des marqueurs de travail en cours, fichiers source vides, chemins d'appel des fonctionnalités et surfaces sensibles.
- Compilation complète de l'environnement `esp32doit-devkit-v1` et tentative d'exécution du test PlatformIO ciblé.

Limites : pas de revue de schéma électronique, mesure de timing, fuzzing, test d'intrusion, audit du service cloud, analyse de dépendances/CVE, tests HIL, ni vérification de la configuration eFuse/NVS sur un appareil. Le rapport est une revue de code, pas une certification de sûreté fonctionnelle.

3. Constats priorisés

P1 — BLOQUANT Commandes MQTT de contrôle sans validation d'origine ni anti-rejeu applicatif

Le gestionnaire accepte des commandes `STOP`, `KILL` et `ENGINE_ON` après vérification du topic et extraction de `id/command`. Il n'exige ni signature/MAC de la commande, ni nonce/compteur, ni horodatage vérifié. La fenêtre `COMMAND_VALIDITY_WINDOW_S` est déclarée mais non appliquée. Les commandes valides sont soumises au gestionnaire du relais ; `ENGINE_ON` permet également un engagement distant. QoS/TLS et identifiants MQTT protègent le transport et la connexion, mais ne remplacent pas cette validation métier ni la protection contre les doublons/rejeux.

Impact : si une commande non autorisée ou ancienne atteint l'abonné (par exemple à la suite d'un défaut d'ACL, de compte ou de traitement), elle peut modifier l'état du relais et du système. La politique de sûreté des commandes `ENGINE_ON` doit aussi être validée par le responsable produit/sûreté.

À faire : imposer un enveloppe signé ou MACé, séquence persistante/nonce et date bornée, contrôle d'autorisation par action, déduplication d'identifiant, journalisation des rejets et essais de rejeu/fuzzing. Définir explicitement l'état sûr et les conditions physiques autorisant toute commande de relais.

Références : `src\services\mqtt\mqtt_command_handler.cpp:76-128` ; `include\config\security_config.h:157` ; Documents 08 §2/§7 et 11 §2.

P1 — BLOQUANT Badge RFID par défaut partagé et enrôlé automatiquement

Au démarrage, tout boîtier déjà provisionné ajoute à sa liste d'autorisation le même UID littéral `91:51:E2:06` si celui-ci n'y figure pas. Ce badge est donc une identité commune prévisible à l'échelle des installations utilisant ce firmware et suffit à être reconnu comme autorisé localement.

À faire : supprimer l'enrôlement automatique, imposer une procédure d'association propre à chaque appareil sous contrôle physique, et vérifier la liste initiale et la révocation en production. Traiter le badge déjà diffusé comme compromis s'il a été utilisé sur des appareils réels.

Références : `src\main.cpp:165,187-201,406`.

P1 — BLOQUANT Le cycle de contrôle peut être bloqué par les opérations réseau

`Application::start()` planifie l'update système toutes les 100 ms. Le scheduler est une tâche FreeRTOS épinglée sur le cœur réseau ; son callback exécute `SystemController::update()`, qui traite aussi le relais. En mode hors ligne, l'update appelle synchroniquement la reconnexion cellulaire/MQTT. Celle-ci contient des échanges AT et des délais de plusieurs secondes (l'établissement MQTT attend jusqu'à 60 s). Le même ordonnanceur n'exécute alors plus son cycle de contrôle pendant l'attente.

Impact : délai ou absence de traitement des événements, du relais, des capteurs et des tâches planifiées ; le comportement réel dépend des priorités, du watchdog et du modem. Ce couplage contredit le modèle de tâches documenté et n'est pas acceptable sans analyse temporelle HIL.

À faire : isoler les appels réseau bloquants dans une tâche de connectivité dédiée, communiquer par files/événements non bloquants, garder le contrôle relais indépendant et démontrer au banc les latences maximales, priorités, watchdog et comportement de panne.

Références : `src\app\application\application.cpp:54-70` ; `src\core\scheduler\scheduler.cpp:24-55` ; `src\app\system_controller\system_controller.cpp:223,404,703-730` ; Document 02 §3.

P1 — BLOQUANT RELEASE Secure Boot et chiffrement flash déclarés obligatoires, mais non activés/vérifiés par ce dépôt

Les constantes déclarent ces protections requises en production, mais `platformio.ini` ne fournit pas de configuration de release/eFuse correspondante et le chemin de démarrage n'appelle pas les fonctions de détection ni le contrôle `bootSecurityStateOk()`. Le test unitaire de ce prédicat ne prouve pas l'activation sur l'appareil.

Conclusion prudente : l'état des eFuses et de l'image réellement programmée est inconnu ; le code/configuration audités ne garantissent pas le refus de démarrer lorsque les protections exigées manquent. La protection des secrets NVS dépend en outre de cette configuration matérielle.

À faire : documenter et automatiser une chaîne de provisioning/release sécurisée, activer les protections par configuration usine officielle, vérifier leur état au démarrage et établir une procédure de récupération sans exposer de clés.

Références : `include\config\security_config.h:126-149` ; `platformio.ini` ; absence d'appel à `bootSecurityStateOk()` dans `src\`.

P2 — ÉLEVÉ Validation temporelle des certificats TLS neutralisée par configuration

La séquence de connexion SIM7600 configure explicitement `ignorelocaltime=1`. Cela demande au modem d'ignorer l'heure locale lors de la validation TLS ; la validité temporelle du certificat serveur (notamment expiration/not-yet-valid) n'est donc pas une garantie du chemin actuel. Le code synchronise l'horloge dans certaines circonstances, ce qui rend cette exception à justifier et à tester.

À faire : retirer cette option si le modem le permet ; sinon documenter précisément les vérifications restantes, la chaîne CA, le nom d'hôte/SNI et les limites, puis tester certificats expirés, mauvais hôte et horloge invalide sur le modèle de modem retenu.

Références : `include\config\sim7600_mqtt_config.h:20-21` ; `src\middleware\mqtt_client\cellular_mqtt_client.cpp:98-100`.

P2 — ÉLEVÉ La télémétrie hors ligne ou échouée n'est pas mise en backlog durable

La télémétrie est envoyée à une file RAM bornée (4 éléments prioritaires et 4 normaux). La tâche de publication consomme le message, tente une publication, puis ne le remet pas en file et ne le transmet pas à la SD en cas d'échec. Le service de stockage observé journalise des enregistrements applicatifs, mais n'est pas branché comme file de reprise de télémétrie. Cela contredit l'exigence offline-first et de rejeu séquentiel des Documents 02/08/09.

À faire : persister la télémétrie avant acquittement cloud, borner la rétention, dédupliquer par séquence, rejouer dans l'ordre et tester coupure réseau/plein SD/redémarrage/coupure d'alimentation.

Références : `src\services\telemetry\telemetry_service.cpp:57-80,157-193` ; `src\services\mqtt\mqtt_publish_queue.h:24-25` ; `src\services\mqtt\mqtt_publish_task.cpp:88-104`.

P2 — ÉLEVÉ OTA signée présente en briques, mais aucun chemin de mise à jour complet relié

Le gestionnaire possède un backend Update et un vérificateur Ed25519, et la clé publique est chargée au démarrage. Cependant, le service OTA est vide et aucune autre partie du firmware ne déclenche la séquence `begin/write/finish` après une commande/manifeste téléchargé. La présence d'un partitionnement OTA ne constitue donc pas une fonctionnalité OTA de bout en bout.

À faire : soit exclure/désactiver l'OTA en production jusqu'à sa réalisation, soit fournir le protocole de téléchargement, vérification de manifeste et version, validation de signature, limites de taille, gestion d'échec/rollback et tests négatifs sur matériel. Vérifier également l'algorithme de signature convenu avec l'outil de signature.

Références : `src\services\ota\ota_service.cpp` (vide) ; `src\managers\ota_manager\ota_manager.cpp:104-135` ; appels observés limités à `init()` dans `src\app\system_controller\system_controller.cpp:74`.

P2 — ÉLEVÉ Les tests PlatformIO ne sont pas exécutables dans la configuration actuelle

Deux fichiers de tests existent (sécurité et timer). La commande ciblée `python -m platformio test -e esp32doit-devkit-v1 -f test_timer --without-uploading` échoue à l'édition de liens avec des symboles Arduino `setup()`/`loop()` manquants ; aucun test n'a été exécuté. L'environnement défini est la cible ESP32 et non un environnement `native` configuré pour les tests unitaires. Les scénarios HIL, sécurité, relais, MQTT et stockage demandés par le Document 11 ne sont pas représentés par les tests présents.

À faire : ajouter une configuration PlatformIO de test native ou un target de test isolé, faire compiler/exécuter les tests en CI, puis compléter par tests des commandes, de l'ordonnanceur, de l'OTA, du stockage et les essais HIL documentés.

Références : `platformio.ini` ; `test\test_security\test_security.cpp` ; `test\test_timer\test_timer.cpp` ; Document 11 §§2-5.

P2 — ÉLEVÉ Contrat MQTT divergent entre spécification et implémentation

Le Document 08 décrit les topics `fortico/<device_id>/...` et un ensemble `cmd/ota/lwt`. Le code construit `fleet/<vehicleId>/<channel>`, s'abonne à la commande par véhicule et publie des canaux différents. Cette divergence peut empêcher l'interopérabilité backend, l'application des ACL et la réception des commandes. Le document marque déjà plusieurs éléments comme hypothèses à valider.

À faire : obtenir un contrat backend signé, choisir identifiant d'appareil ou véhicule comme racine, uniformiser topics/QoS/retain/payloads, et ajouter des tests de contrat automatisés partagés.

Références : `src\services\mqtt\mqtt_topics.cpp:11-23` ; `src\middleware\mqtt_client\cellular_mqtt_client.cpp:188-250` ; Document 08 §§3-7.

P3 — À PLANIFIER **Modules vides et architecture partiellement matérialisée**

Douze fichiers source/header sous `src\` sont vides, dont les modules nommés `NetworkService`, `OtaService`, `TrackingManager`, `TopicRouter` et l'ancien `GpsDriver`. Plusieurs composants prévus par les documents ne sont donc que déclarés par leur emplacement ou remplacés par des implémentations alternatives dans d'autres modules. La compilation ne détecte pas ce manque fonctionnel.

À faire : marquer explicitement les composants hors périmètre, retirer les stubs inutilisés ou les implémenter, et mettre à jour l'architecture/documentation pour refléter le chemin réellement exécuté.

Références : fichiers vides sous `src\services\network\`, `src\services\ota\`, `src\services\mqtt\`, `src\managers\tracking_manager\` et `src\drivers\gps\`.

4. Éléments positifs et état technique

- Organisation en couches lisible (HAL, drivers, managers, services, core et application) et injection explicite des dépendances sur les principaux chemins.
- Compilation de production PlatformIO réussie avec C++17, `-Wall -Wextra -Werror`. Taille observée : 616 209 octets flash, soit 47 % de la partition applicative annoncée (1 310 720 octets), et 29 860 octets RAM statique, soit 9,1 % du total indiqué. Ces mesures ne représentent pas le heap libre en fonctionnement.
- Identifiants MQTT chiffrés par AES-GCM au niveau applicatif ; les UID autorisés sont stockés sous forme de HMAC plutôt qu'en clair dans la liste NVS. Ces mesures restent dépendantes de la protection effective de la clé racine et de la flash.
- Le parser SIM7600 borne les longueurs des topics/payloads entrants à la taille des buffers avant traitement ; la file MQTT est également bornée.

La compilation prouve que le code cible se compile ; elle ne prouve ni le bon fonctionnement des transactions AT, ni la configuration TLS/ACL, ni les temps de réaction du relais, ni l'intégrité après perte d'alimentation.

5. Plan de remédiation recommandé

Ordre	Action de sortie	Critère de clôture
1 — Sécurité d'accès	Supprimer le badge commun ; appliquer authentification/autorisation/anti-rejeu aux commandes MQTT et définir l'autorisation des commandes de relais.	Tests négatifs signés, expirés, rejoués, doublonnés et non autorisés ; revue sûreté validée.
2 — Sûreté/temps réel	Déplacer les reconnexions et transactions réseau bloquantes hors du cycle de contrôle du relais.	Mesures HIL sous perte réseau et modem bloqué ; aucune tâche critique affamée, latence et watchdog sous seuil documenté.
3 — Configuration de production	Définir provisioning, Secure Boot, chiffrement flash/NVS et politique TLS/certificats pour les cartes de production.	Build reproductible de release ; vérification au boot et preuve de configuration eFuse sur matériel d'essai.
4 — Qualité et tests	Réparer l'environnement de tests et automatiser compilation/test dans CI ; compléter les tests de sécurité et de sûreté du Document 11.	Tests exécutés et tracés ; échecs de compilation/tests bloquent le merge.

Ordre	Action de sortie	Critère de clôture
5 — Continuité et OTA	Créer backlog SD durable et rejeu déterministe ; compléter OTA ou désactiver son annonce et son déclenchement.	Scénarios panne réseau/redémarrage, OTA invalide/valide/rollback passés sur banc.
6 — Contrat produit/backend	Aligner topics, payloads, QoS, identités, durée de validité et commandes avec le backend et les guides d'architecture.	Tests de contrat communs et documentation versionnée.

6. Vérifications exécutées

Vérification	Résultat	Interprétation
`python -m platformio run -e esp32doit-devkit-v1`	**Réussie**	Build firmware ESP32 finalisé sans erreur.
`python -m platformio test -e esp32doit-devkit-v1 -f test_timer --without-uploading`	**Échouée**	Édition de liens : références `setup()`/`loop()` absentes ; 0 test exécuté.
Inspection du dépôt/documentation	**Effectuée**	Configuration, sources ciblées, tests et spécifications lus ; 12 fichiers vides identifiés sous `src\`.
HIL, broker et configuration eFuse	**Non exécutés**	Matériel, broker et procédure de fabrication non accessibles dans ce contexte.

Les commandes de build/test ont été lancées avec Python PlatformIO Core 6.2.0. Le test de l'ensemble avait aussi échoué pendant une tentative concurrente build/test ; le test timer isolé a ensuite reproduit proprement l'erreur de liens. Aucun test ne doit être considéré comme passé.

7. Conclusion

Le firmware est compilable et possède des composants de sécurité utiles, mais les divergences entre exigences documentées et chemins réellement exécutés sont significatives. La recommandation est de bloquer la mise en service en flotte jusqu'à clôture des constats P1, validation des propriétés TLS/flash sur matériel, rétablissement des tests automatisés et exécution des scénarios HIL critiques. Les constats P2 concernant l'OTA, le backlog et le contrat MQTT doivent aussi être résolus ou explicitement exclus du périmètre avant d'annoncer ces fonctions.

Fin du rapport — Revue basée sur l'état du dépôt visible au 6 novembre 2026.